

Lab 2: Linear Regression Model

In this lab you will:

- Explore scikit-learn, a popular Python library for machine learning that offers a variety of algorithms and preprocessing functions.
- Use scikit-learn to preprocess input data for regression tasks, including techniques such as data normalization and text embedding.
- Build a simple regression model with scikit-learn to predict ice cream sales based on temperature.
- Implement a linear regression model with scikit-learn to predict crop yields. Given input data including crop type, planting region, temperature, and precipitation, you will forecast crop yield.

scikit-learn

The scikit-learn is a machine learning library for python. It includes several regression, classification, cluster methods and many preprocessing and evaluation methods.

The homepage of scikit-learn provides many tutorials about its usages and sub-modules. You can access the website to learn more about scikit-learn: <https://scikit-learn.org/stable/index.html> .

The scikit-learn provides many sub-modules for machine learning. Generally speaking, for nearly every type of machine learning, it provides a corresponding sub-module. For example, the `sklearn.linear_model` provides implementation of linear regression and classification.

Linear classifiers

<code>LogisticRegression</code>	Logistic Regression (aka logit, MaxEnt) classifier.
<code>LogisticRegressionCV</code>	Logistic Regression CV (aka logit, MaxEnt) classifier.
<code>PassiveAggressiveClassifier</code>	Passive Aggressive Classifier.
<code>Perceptron</code>	Linear perceptron classifier.
<code>RidgeClassifier</code>	Classifier using Ridge regression.
<code>RidgeClassifierCV</code>	Ridge classifier with built-in cross-validation.
<code>SGDClassifier</code>	Linear classifiers (SVM, logistic regression, etc.) with SGD training.
<code>SGDOneClassSVM</code>	Solves linear One-Class SVM using Stochastic Gradient Descent.

Classical linear regressors

<code>LinearRegression</code>	Ordinary least squares Linear Regression.
<code>Ridge</code>	Linear least squares with L2 regularization.
<code>RidgeCV</code>	Ridge regression with built-in cross-validation.
<code>SGDRegressor</code>	Linear model fitted by minimizing a regularized empirical loss with SGD.

In addition, it also provides several sub-modules related to machine learning. The `sklearn.preprocessing` provides many data preprocessing function for machine learning including scaling, centering, normalization, binarization, and more. The `sklearn.model_selection` sub-module provides tools for dataset split and cross validation. The `sklearn.metrics` can help to calculate metrics including MSE, MAE and R2 scores to evaluate machine learning models. All sub-modules' detailed introduction can be found in the website: <https://scikit-learn.org/stable/api/index.html>.

The anaconda has already included scikit-learn. If not, you can follow the command to install:

```
pip install -U scikit-learn
```

or using conda tool to install:

```
conda install scikit-learn
```

Exercise 1: Learn how to use scikit-learn to predict ice cream sales according to the temperature using liner regression model

Given the temperature values, this task is to predict the scales of ice cream. Ice cream sales are generally proportional to temperature. In this lab, we utilize scikit-learn to build a linear regression model to get the mapping function from temperature to sales:

$$scales = F(temperature)$$

It is a simple regression problem with only one input variable. The data used in this lab is in folder “ice_cream_sales”:

Temperature(degree)	IceCreamSales(dollar)
14.2	215
16.4	325
11.9	185
15.2	332
18.5	406
22.1	522
19.4	412
25.1	614
23.4	544
18.1	421
22.6	445
17.2	408

In the following instructions, we will show you how to read data, display data, build linear regression model and evaluate the model.

1) Create new python file named “ice_cream_sales_regression.py”

Following the instructions in lab 1 to create a new project and new python file for this lab.

2) Import some python libraries for next processing

matplotlib.pyplot for displaying data and results, sklearn.linear_model for linear regression model, sklearn.metrics for evaluation, numpy for data load and processing.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

3) Read csv data

Just like what we did in the lab 1 section 2.2 to load csv data with numpy library. The first column is temperature, while the second column is ice cream sale

```
data_file=r'ice_cream_sales/ice.csv'
data=np.loadtxt(data_file, delimiter=',', skiprows=1)
temperature=data[:, 0:1]
sales=data[:, 1:2]
```

Note that the slash “/” in the data path, not the “\”, unless you add an “r” in front of it.

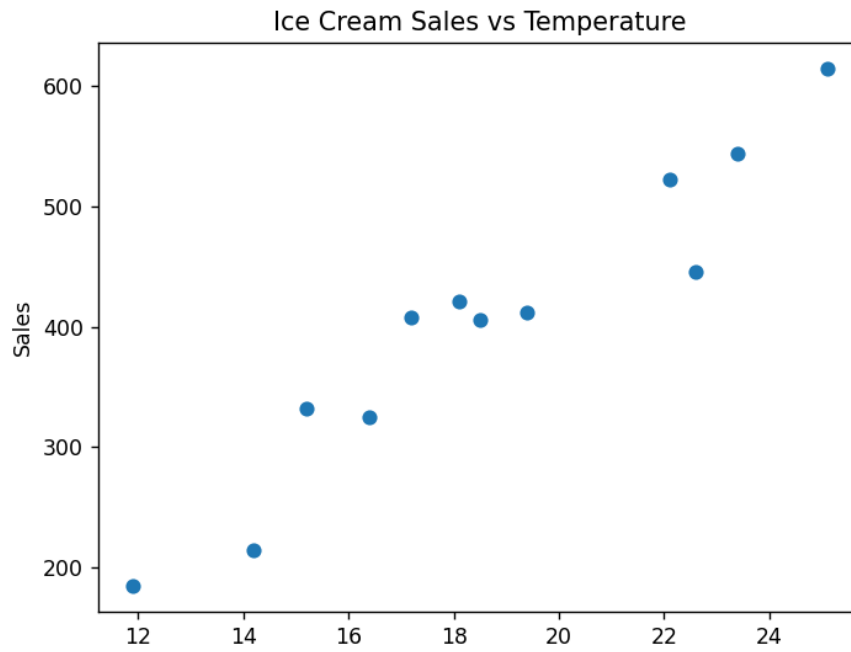
4) display temperature and sales data in a scatter plot

Now, we will display the temperature data and sale data in a scatter plot to show their relationship.

```
plt.scatter(temperature, sales)      #scatter plot
plt.xlabel('Temperature')           #x-axis label
plt.ylabel('Sales')                 #y-axis label
plt.title('Ice Cream Sales vs Temperature') #title
plt.show()                          #display the plot
```

If it does not display properly, you can select file->settings. Enter “python plots” in the search box and uncheck "show plot in tool window" on the right. Then click ok and run the program again to get the visualization.

Here is the scatter plot.



5) build linear regression model using scikit-learn

Obviously, they have a roughly linear relationship. Next, we use scikit-learn to build a linear regression model: $y = ax + b$, where x represents the temperature and b represents ice cream prices. We don't need to calculate manually, but use scikit-learn's method `LinearRegression`.

```
sales_model=LinearRegression()
```

6) Fit data to update the parameters in linear regression model

Most models in scikit-learn have several same functions: 1) `fit` is to train the model getting the model's weights. 2) `predict` is to get the output with input data. In this step, we use "fit" method to calculate weights in linear regression model.

```
sales_model.fit(temperature, sales)
```

7) Print model parameters

After trained, we can get and print the parameters from model

```
a=sales_model.coef_[0][0]  
b=sales_model.intercept_[0]  
print('a: {0}, b: {1}'.format(a, b))
```

8) predict sales using the model

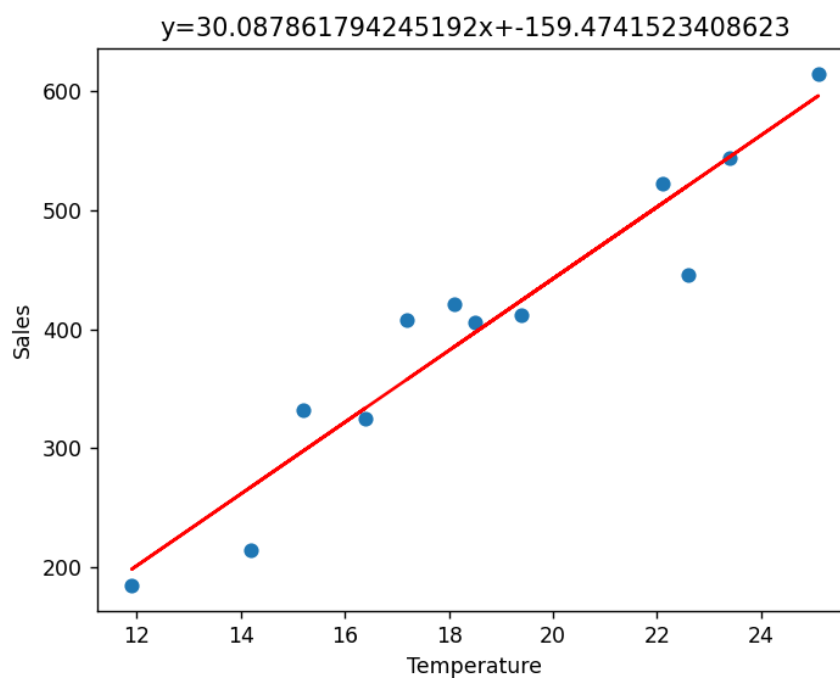
using the trained model to predict the sales with predict method

```
sales_pred=sales_model.predict(temperature)
```

9) display the regression line

Display the regression line using plot method in matplotlib.

```
plt.plot(temperature, sales_pred, color='red') #regression line  
plt.scatter(temperature, sales) #scatter plot  
plt.xlabel('Temperature') #x-axis label  
plt.ylabel('Sales') #y-axis label  
plt.title('y={0}x+{1}'.format(a, b)) #title  
plt.show()
```



10) Evaluate the linear model and calculate metrics

Calculate mse, mae and r2 scores to evaluate the model

```
mse=mean_squared_error(sales, sales_pred)  
r2=r2_score(sales, sales_pred)  
mae=mean_absolute_error(sales, sales_pred)
```

```
print('MSE: {0}, R2: {1}, MAE: {2}'.format(mse, r2, mae))
```

Exercise 2: Learn how to use scikit-learn to achieve regression task crop yield prediction, including input data preprocessing, model built and evaluation

It is a more complicated problem. In this lab, some meteorological and crop variables are provided to predict the crop yield.

It is a multi-variable regression problem. Although it is still a linear regression model, multi-variables should be considered. The model parameters we should calculate are vectors of the same length as the number of variables. The used data is saved in folder “crop_yield”.

Area	Item	Year	average_rain_fall_mm_per_year	pesticides_tonnes	avg_temp	hg/ha_yield
Albania	Maize	1990	1485	121	16.37	36613
Albania	Potatoes	1990	1485	121	16.37	66667
Albania	Rice, paddy	1990	1485	121	16.37	23333
Albania	Sorghum	1990	1485	121	16.37	12500
Albania	Soybeans	1990	1485	121	16.37	7000
Albania	Wheat	1990	1485	121	16.37	30197
Albania	Maize	1991	1485	121	15.36	29068
Albania	Potatoes	1991	1485	121	15.36	77818
Albania	Rice, paddy	1991	1485	121	15.36	28538
Albania	Sorghum	1991	1485	121	15.36	6667
Albania	Soybeans	1991	1485	121	15.36	6066
Albania	Wheat	1991	1485	121	15.36	20698
Albania	Maize	1992	1485	121	16.06	24876
Albania	Potatoes	1992	1485	121	16.06	82920
Albania	Rice, paddy	1992	1485	121	16.06	40000
Albania	Sorghum	1992	1485	121	16.06	3747
Albania	Soybeans	1992	1485	121	16.06	4507
Albania	Wheat	1992	1485	121	16.06	24388
Albania	Maize	1993	1485	121	16.05	24185
Albania	Potatoes	1993	1485	121	16.05	98446
Albania	Rice, paddy	1993	1485	121	16.05	41786
Albania	Soybeans	1993	1485	121	16.05	7998
Albania	Wheat	1993	1485	121	16.05	29976
Albania	Maize	1994	1485	201	16.96	25848
Albania	Potatoes	1994	1485	201	16.96	81404
Albania	Soybeans	1994	1485	201	16.96	7927
Albania	Wheat	1994	1485	201	16.96	24745
Albania	Maize	1995	1485	201	15.57	21200

Data in column “hg/ha_yield” records the crop yield in hundred grams per hectare. It is utilized as the predict target. Other columns represent record regions, crop types, precipitation per year in millimeters, pesticides in tones and average temperature. These variables are the input of linear regression model.

Thanks to scikit-learn for its ease of use, we can follow the same way to train the linear regression model, even if the number of input variables differs.

However, it can be noted that in this task, the input data not only has numerical values, but also has two columns of text data, and these two text

variables are very important for yield estimation. In addition, different data variables have different ranges, which will affect their influence in the linear regression model. We need to transform them to the same range.

Therefore, the key is how to preprocess the data to ensure that it can be correctly input into the model. This is also an important step in machine learning/deep learning applications.

- 1) Create new python file named “crop_yield_prediction.py”
- 2) Import python libraries

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score,
mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder
```

Except the libraries which we have used in the exercise 1, we import two extra sub-modules from scikit-learn. “train_test_split” is used to split the training and testing dataset. The training dataset is used to train the model, while test dataset is used to evaluate the model. “MinMaxScaler” is used to normalize the training and testing data and convert them into range with [0,1].

3) Load and split data

We need to load numerical data and text data, respectively. The `numpy.loadtxt` can assign specific columns we want to load.

```
data_file=r'crop_yield/yield_data.csv'
```

```
#load the numerical data
```

```
numerical_data=np.loadtxt(data_file, delimiter=',', skiprows=1, usecols=(2,3,4,5))
```

It is to load csv data in column from 2 3 4 5, which are all numerical data

```
#load the text data
```

```
text_data=np.loadtxt(data_file, delimiter=',', skiprows=1, usecols=(0,1),
dtype='str')
```

Load text data from first two columns, with type string.

```
#load the target data
```

```
target_data=np.loadtxt(data_file, delimiter=',', skiprows=1, usecols=(6))
```

4) convert the text data into one-hot encoding

One-Hot Encoding is a process of converting categorical data variables so they can be provided to machine learning algorithms to improve predictions. With one-hot, we convert each categorical value into a new categorical column and assign a binary value of 1 or 0. Each integer value is represented as a binary vector. All the values are zero, and the index is marked with a 1.

Take variable “crop type” as an example. Suppose variable "Crop_Type" having values as "Corn", "Wheat", "Soybeans" and “rice”, after one hot encoding, it seems like matrix:

corn	wheat	soybean	rice
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

Although it looks complex, it becomes easy if we use scikit-learn. Just use the sub-module OneHotEncoder to convert the text data into one hot embedding.

```
text_encoder=OneHotEncoder()  
text_encoder.fit(text_data)  
text_data_onehot=text_encoder.transform(text_data).toarray()  
print(text_data_onehot.shape)
```

As the “Item” column contains 10 different values and “Area” column has 101 different values, the converted one hot vector for each sample is with size of 111.

5) combine the one-hot encoded text data and numerical data, and split the data into training and testing sets

Now, we need to combine the converted one hot data and numerical data. At the same time, we can also use “train_test_split” to split the data into training and testing sets. The test set is used to simulate real application scenarios and reflect the application performance of the model.

```
input_data=np.concatenate((text_data_onehot, numerical_data), axis=1)  
train_input, test_input, train_target, test_target = train_test_split(input_data,  
target_data, test_size=0.25)
```

6) normalize the input data

Utilize “MinMaxScaler” to normalize the input data to range [0,1].

```
scaler=MinMaxScaler()  
scaler.fit(train_input) # calculate the min and max value of each variables in the  
training set  
train_input=scaler.transform(train_input) # normalize the training set  
test_input=scaler.transform(test_input) # normalize the testing set
```

7) Build and train model

After the data preprocessing, the model build and evaluation is the same with exercise 1.

```
yield_model=LinearRegression()  
yield_model.fit(train_input, train_target)
```

8) Evaluate the model

```
target_pred=yield_model.predict(test_input)  
mse=mean_squared_error(test_target, target_pred)  
r2=r2_score(test_target, target_pred)  
mae=mean_absolute_error(test_target, target_pred)  
print('MSE: {0}, R2: {1}, MAE: {2}'.format(mse, r2, mae))
```